Tic-Tac-Toe Full-Stack AI Project

C++ Engine + Python + FastAPI + React

A learning-first implementation plan

Purpose

This project is a small system for learning the complete path from a performance-oriented C++ engine to a Python research layer and API, then to a React visualization. Tic-Tac-Toe is intentionally simple so the focus can remain on architecture, interfaces, algorithms, testing, language boundaries, and system integration.

The project is not about producing the most elaborate Tic-Tac-Toe game possible. It is preparation for building larger game-AI systems later.

Core learning rule

If I am learning it, AI tutors me. If I already understand it and it is tedious, AI can implement it.

Do not ask AI to make the important design decisions before attempting them yourself. Use AI to critique decisions, provide hints, explain unfamiliar syntax, generate repetitive scaffolding, and accelerate frontend work.

Target Architecture

```
+-------------------------------------------------+
| React + TypeScript |
| |
| Board UI Game status AI analysis |
+-------------------------+-----------------------+
                          |
                    HTTP / JSON
                          |
                          v
+-------------------------------------------------+
| Python + FastAPI |
| |
| API routes | validation | serialization |
| experiments | orchestration |
+-------------------------+-----------------------+
                          |
                       pybind11
                          |
                          v
+-------------------------------------------------+
| C++ Engine |
```

```
| |
| Board | rules | Minimax | Alpha-Beta | metrics |
+-------------------------------------------------+
```

Architectural rule: C++ is the source of truth for the game. Python and React must not duplicate the Tic-Tac-Toe rules. Python orchestrates and exposes the engine. React presents state and sends user actions.

Phase 1: Design and Build the C++ Engine

Objective: Build a standalone C++ game engine that works without Python, a web server, or a browser.

Before implementation, decide the responsibilities and interfaces for the major concepts. Possible concepts include `Board`, `Game`, `Move`, `Player`, and eventually `Agent`. The exact design should be your decision.

Questions to answer before coding:

- Who owns the current turn?
- Who determines whether a move is legal?
- Who determines whether the game has ended?
- How is a move represented?
- How are board states copied or modified during search?
- What information should an AI agent receive?
- Which objects should know about players, and which should not?

AI usage: low. Write the initial architecture and interfaces yourself. Good prompts ask AI to critique your proposed separation of responsibilities, identify coupling, or explain a C++ concept. Avoid asking AI to design the entire engine.

Completion checkpoint: A command-line program can play a complete legal game, detect wins/draws, reject illegal moves, and operate with no Python or web code.

Phase 2: Implement Minimax and Alpha-Beta Pruning

Objective: Learn recursive game-tree search by implementing it yourself.

Use a simple terminal evaluation such as

$$V(s) = \begin{cases} +1 & \text{AI win} \\ 0 & \text{draw} \\ -1 & \text{AI loss.} \end{cases}$$

For each nonterminal state, generate legal actions, construct the resulting state, recursively evaluate it, and select the maximum or minimum value depending on whose turn it is.

Implementation progression:

1. Implement plain Minimax.
2. Verify that the agent cannot lose when playing optimally.
3. Count nodes searched for each decision.
4. Implement alpha-beta pruning.
5. Compare node counts and execution time against plain Minimax.

AI usage: very low. Do not ask AI to generate the initial Minimax or alpha-beta implementation. Use it for conceptual hints, review of your reasoning, or debugging clues after you have made a real attempt.

Completion checkpoint: Minimax plays optimally, alpha-beta produces the same decisions, and you can explain why pruning reduces the amount of search.

Phase 3: Build a C++ Test Suite

Objective: Make the engine trustworthy before exposing it to other languages.

Use GoogleTest and cover behavior such as:

- An empty board has nine legal moves.
- A legal move changes the expected square.
- An occupied square cannot be overwritten.
- Horizontal, vertical, and diagonal wins are detected.
- Draws are detected correctly.
- Minimax takes an immediate winning move.
- Minimax blocks an immediate loss.
- Minimax does not lose against weaker agents.

AI usage: medium-high after you specify the tests. You decide which behaviors and edge cases matter. AI may generate repetitive GoogleTest scaffolding or help diagnose CMake/test configuration problems.

Completion checkpoint: The engine and search algorithms have automated tests that run from the build system and pass consistently.

Phase 4: Expose the Engine to Python with pybind11

Objective: Learn how a Python process can call compiled C++ code.

Expose only a clean public interface. A Python user should eventually be able to do something conceptually similar to:

```
import tictactoe_engine

board = tictactoe_engine.Board()
board.make_move(4, "X")
print(board.legal_moves())

agent = tictactoe_engine.MinimaxAgent()
move = agent.best_move(board)
```

Understand the boundary:

```
Python object
     |
     v
pybind11 binding
     |
     v
C++ object / method
```

Pay attention to type conversion, ownership, references, object lifetime, exceptions, and which parts of the C++ API should actually be exposed.

AI usage: medium-high. Learn the purpose and mechanics of bindings yourself, then use AI to help with pybind11 syntax, build configuration, and repetitive binding declarations.

Completion checkpoint: Python can create game states, request legal moves, make moves, and invoke the C++ AI without reimplementing game logic in Python.

Phase 5: Use Python for Experiments

Objective: Establish a clean division between high-performance simulation and flexible experimentation.

Use C++ for game execution and search. Use Python to configure experiments, collect results, analyze data, and create plots.

Run matchups such as:

- Random vs. Random
- Random vs. Minimax
- Random vs. Alpha-Beta
- Minimax vs. Minimax

Collect measurements such as win/draw/loss rates, nodes searched, decision time, and games per second.

```
C++ Python
------------------- --------------------
Game rules Experiment setup
Search Repeated trials
Simulation Result collection
Performance-critical work Statistics and plots
```

AI usage: medium. Write the experimental questions and metrics yourself. AI can help with plotting, dataframe manipulation, repetitive experiment harness code, or statistical explanations.

Completion checkpoint: A Python script can run large batches of games through the C++ engine and produce useful quantitative comparisons between agents.

Phase 6: Add a FastAPI Backend

Objective: Turn the Python layer into a network-accessible service.

Keep the first API small. Possible endpoints include:

```
POST /games
GET /games/{id}
POST /games/{id}/moves
GET /games/{id}/analysis
```

A move request should travel through the entire stack:

```
HTTP request
    |
    v
FastAPI route
    |
    v
Python orchestration
    |
    v
pybind11
    |
    v
C++ game engine
    |
    v
Python serialization
    |
    v
JSON response
```

Learn what HTTP methods mean, how request/response schemas work, how errors are represented, and where validation belongs.

AI usage: medium. Design the API yourself first. AI can help implement FastAPI boilerplate, Pydantic models, serialization, and routine endpoint code after the interface is defined.

Completion checkpoint: The game can be played entirely through HTTP requests while C++ remains the source of truth for game state and rules.

Phase 7: Build the React + TypeScript Frontend

Objective: Build a visual client for the API without duplicating backend logic.

Learn enough React and TypeScript to understand components, props, state, events, asynchronous HTTP requests, and the flow of data through the UI.

A simple component structure could resemble:

```
App
 |
 +-- GameBoard
 | +-- Square
 |
 +-- GameStatus
 |
 +-- AIAnalysis
```

The frontend should ask the API whether a move is valid rather than maintaining an independent copy of the game rules.

AI usage: high. You should understand and choose the component/data-flow architecture, but AI can do much of the React implementation, CSS, responsive layout, API-client boilerplate, and visual polish. Review generated code and make sure you can trace how user actions become API requests and state updates.

Completion checkpoint: A user can play against the C++ AI through the browser, and you can explain the full path from a click in React to a C++ method call and back.

Phase 8: Visualize AI Decisions

Objective: Turn the frontend into an AI-inspection tool rather than only a game board.

Extend the C++ search API to return useful analysis metadata, for example:

```
{
  "best_move": 4,
  "nodes_searched": 1832,
  "evaluation": 1,
  "move_scores": [
    {"position": 1, "score": -1},
    {"position": 4, "score": 1},
    {"position": 7, "score": 0}
  ]
}
```

Possible visualizations include:

- Score for every legal move.
- Selected best move.
- Number of nodes searched.
- Minimax evaluation of the current position.
- Comparison of Minimax and alpha-beta node counts.
- A small, bounded visualization of part of the search tree.

AI usage: high for visualization implementation, low for deciding what the algorithm should report. You define meaningful metrics. AI can help build charts, board overlays, animations, and presentation components.

Completion checkpoint: The website makes the AI's decision process inspectable rather than merely showing the move it selected.

Phase 9: Package with Docker

Objective: Learn how the compiled engine, Python extension, API, and frontend can be built and run reproducibly.

Before generating a Dockerfile, understand the local build chain:

```
C++ source
    |
    v
CMake / compiler
    |
    v
C++ library + Python extension
    |
    v
Python dependencies
    |
    v
FastAPI server
    |
    v
React client
```

Only after the local system works should it be containerized. Avoid adding unrelated infrastructure such as Kubernetes, Redis, authentication, or a database unless a real project requirement appears.

AI usage: medium. Understand every Docker stage and command. AI may help create or simplify the Dockerfile, diagnose build failures, and reduce repetitive configuration.

Completion checkpoint: The application can be built and run from a clean environment using documented commands, with the C++ extension compiled correctly.

AI Usage Guide

Area	AI level	Learning rule
Architecture	Low	Propose the design yourself; ask AI for critique.
C++ class design	Low	Own responsibilities, interfaces, and tradeoffs.
Minimax	Very low	Implement and reason through the recursion yourself.
Alpha-beta	Very low	Derive and implement after understanding Minimax.
Debugging	Medium	Ask for diagnosis/hints before requesting a patch.
CMake	Medium	Understand targets and dependencies; use help for syntax.
GoogleTest	High	You specify behaviors; AI may write scaffolding.
pybind11	Medium-high	Understand the boundary; use help for binding syntax.
Python experiments	Medium	Own questions/metrics; automate repetitive analysis.
FastAPI	Medium	Design endpoints first; use AI for boilerplate.
React architecture	Medium	Own component/data flow.
React implementation	High	Use AI heavily after defining behavior.
CSS / visual polish	Very high	Delegate most implementation while reviewing output.
Docker	Medium	Understand the build chain and every stage.
Documentation	Medium	Use AI to polish explanations you understand.

Recommended Repository Structure

```
tictactoe-ai/
|
+-- engine/
| +-- include/
| +-- src/
| +-- tests/
|
+-- bindings/
|
+-- python/
| +-- experiments/
|
+-- server/
|
+-- web/
|
+-- CMakeLists.txt
+-- Dockerfile
+-- README.md
```

The exact structure can change as the design develops. Do not create abstractions or directories solely because they appear in this plan.

Definition of Done

The project is complete when you can:

- Explain the C++ game architecture and why responsibilities are separated as they are.
- Explain and implement Minimax and alpha-beta pruning.
- Demonstrate automated C++ tests.
- Explain how pybind11 connects Python and C++.
- Run and analyze agent experiments from Python.
- Explain the API contract and request lifecycle in FastAPI.
- Trace a browser click through React, HTTP, Python, pybind11, C++, and back.
- Visualize meaningful search information in the browser.
- Build and run the complete system reproducibly.
- Identify which code was delegated to AI and still explain how the resulting system works.

Final Principle

The purpose of the project is not to prove that you can type every line without assistance. The purpose is to develop enough understanding that you can design the system, make the important technical decisions, verify AI-generated work, debug failures across boundaries, and explain why the finished system works.